

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЁТ ПО УЧЕБНОЙ ОЗНАКОМИТЕЛЬНОЙ
ПРАКТИКЕ

Оценка _____

Студент
гр. БИН-24-1

_____ Е. Е. Фоменко

Руководитель,
ассистент

_____ А. Д. Шнейдер

Руководитель,
профессор

_____ В. М. Гриняк

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

Индивидуальное задание
на учебную ознакомительную практику

Студенту гр. БИН-24-1 Фоменко Егор Евгеньевич

1 Разработать программный продукт «Генератор сводных отчётов из CSV»
(вариант Б-17):

- Загрузка и объединение нескольких CSV-файлов с одинаковой структурой.
- Группировка записей по выбранному полю.
- Вычисление статистических характеристик: количества, суммы, среднего арифметического, медианы, минимального и максимального значений.
- Сортировка сформированных результатов.
- Формирование горизонтальной и процентной ASCII-диаграмм.
- Экспорт сформированного отчёта в форматы CSV и TXT.

2 Соблюдать общие требования к выполнению задания (документ «Варианты индивидуального задания», раздел 2):

- Выбор языка программирования с обоснованием.
- Хранение данных в файлах без СУБД.
- Контейнеризация (Dockerfile).
- Тестирование (юнит- и интеграционные тесты, тестовые данные).
- Отчёт 20–25 страниц по СК-СТО-ТР-04-1.005-2015.
- Репозиторий с README.md.

3 Срок сдачи отчёта на кафедру: 18.07.2026

Руководитель,
ассистент

_____ А. Д. Шнейдер

Руководитель,
профессор

_____ В. М. Гриняк

Задание получил:

_____ Фоменко Егор Евгеньевич

Содержание

Введение	3
1 Анализ задачи и алгоритм	5
1.1 Постановка задачи	5
1.2 Особенности обработки CSV-файлов	5
1.3 Алгоритм группировки и статистической обработки данных	6
1.4 Структуры данных и сортировка результатов	9
2 Обоснование средств разработки и проектирование	10
2.1 Выбор языка программирования и среды разработки	10
2.2 Выбор библиотек и программных средств	11
2.3 Архитектура программного средства	12
3 Реализация программного средства	14
3.1 Реализация алгоритмического ядра	14
3.2 Реализация формирования диаграмм и экспорта	15
3.3 Реализация графического пользовательского интерфейса	16
4 Тестирование и отладка программного средства	18
4.1 Организация тестирования	18
4.2 Модульное и интеграционное тестирование	18
4.3 Анализ результатов тестирования	19
5 Контейнеризация программного средства	20
5.1 Подготовка контейнера приложения	20
5.2 Сборка и запуск программного средства в Docker	20
6 Руководство пользователя	22
6.1 Запуск программного средства	22
6.2 Работа с программой	22
6.3 Экспорт результатов	22
Заключение	24
Список использованных источников	25

Введение

В настоящее время табличные данные широко используются для хранения и обмена информацией между программными системами. Одним из распространённых форматов представления таких данных является CSV. Простота структуры и возможность обработки средствами различных языков программирования позволяют применять данный формат для хранения результатов наблюдений, финансовых показателей, статистических данных и другой структурированной информации.

При работе с несколькими CSV-файлами возникает необходимость объединения содержащихся в них данных, их группировки и последующего статистического анализа. Выполнение данных операций вручную требует значительных временных затрат и повышает вероятность возникновения ошибок. Поэтому разработка программного средства, автоматизирующего формирование сводных отчётов, является практически значимой задачей.

В рамках учебной ознакомительной практики разработано программное средство «Генератор сводных отчётов из CSV». Программа предназначена для загрузки нескольких CSV-файлов, объединения содержащихся в них записей и формирования сводного отчёта на основе выбранных пользователем параметров анализа.

Целью учебной ознакомительной практики является разработка, тестирование и развёртывание прикладного программного продукта для автоматизированного формирования сводных отчётов на основе данных нескольких CSV-файлов.

Для достижения поставленной цели были решены следующие задачи:

- проведён анализ задачи обработки и объединения данных CSV-файлов;
- разработан алгоритм группировки данных и вычисления статистических характеристик;
- выбраны средства разработки и спроектирована модульная архитектура программного средства;
- реализованы загрузка и объединение нескольких CSV-файлов;
- реализованы группировка, статистическая обработка и сортировка результатов анализа;
- разработаны пользовательский интерфейс, средства формирования диаграмм и экспорта отчётов;
- проведено тестирование разработанного программного средства;
- выполнена контейнеризация приложения с использованием Docker;
- подготовлены руководство пользователя и документация по запуску программного средства.

Объектом исследования является процесс обработки и анализа структурированных табличных данных, представленных в формате CSV.

Предметом исследования являются алгоритмы объединения, группировки, статистической обработки и сортировки данных при формировании сводных отчётов.

Практическая значимость разработанного программного средства заключается в автоматизации обработки нескольких CSV-файлов и получении сводной статистической информации без необходимости ручного объединения данных и расчёта показателей.

1 Анализ задачи и алгоритм

1.1 Постановка задачи

При обработке табличной информации данные могут быть распределены между несколькими CSV-файлами, имеющими одинаковую структуру. Для получения общей статистической информации содержащиеся в таких файлах записи необходимо объединить, распределить по группам и выполнить вычисление требуемых показателей. При большом количестве записей ручное выполнение данных операций является трудоёмким и может приводить к ошибкам.

Для автоматизации данного процесса разработано программное средство «Генератор сводных отчётов из CSV», предназначенное для объединения и анализа структурированных данных из нескольких CSV-файлов.

Исходными данными для работы программы являются:

- один или несколько CSV-файлов с одинаковым набором столбцов;
- поле, используемое для группировки записей;
- числовое поле, значения которого используются при статистической обработке;
- параметры сортировки результатов;
- выбранный тип текстовой ASCII-диаграммы.

Результатом работы программы является сводный отчёт, содержащий сформированные группы и рассчитанные для каждой группы статистические характеристики: количество записей, сумму, среднее арифметическое, медиану, минимальное и максимальное значения.

Таким образом, основной задачей программного средства является автоматизация процесса объединения, группировки и статистической обработки данных нескольких CSV-файлов с последующим формированием сводного отчёта в удобной для пользователя форме.

1.2 Особенности обработки CSV-файлов

CSV представляет собой текстовый формат хранения табличных данных, в котором каждая строка соответствует отдельной записи, а значения столбцов разделяются специальным символом-разделителем. Формат широко применяется для обмена структурированной информацией благодаря простоте формирования и обработки файлов.

При объединении нескольких CSV-файлов необходимо учитывать соответствие их структуры. Для корректной совместной обработки загруженные файлы должны содержать одинаковый набор столбцов. Несоответствие структуры может привести к отсутствию необходимых значений или невозможности выполнения группировки по выбранному полю.

В разработанном программном средстве обработка исходных данных выполняется последовательно. После выбора CSV-файла производится его чтение и определение набора заголовков столбцов. После загрузки выбранных CSV-файлов содержащиеся в них записи

последовательно объединяются в общий набор данных, который затем используется для дальнейшей группировки и статистической обработки.

Если структура файлов является совместимой, содержащиеся в них записи могут рассматриваться как единый набор данных. Объединение выполняется путём последовательного добавления записей каждого файла к общей коллекции. При этом исходные CSV-файлы не изменяются.

Пусть имеется m CSV-файлов, каждый из которых содержит набор записей D_i . Объединённый набор данных определяется по формуле (1).

$$D = D_1 \cup D_2 \cup \dots \cup D_m, \quad (1)$$

где D – объединённый набор данных;

D_i – набор записей i -го CSV-файла;

m – количество CSV-файлов.

В данном случае операция объединения означает последовательное включение записей исходных файлов в общий набор данных. Полученная коллекция D используется при дальнейшей группировке и статистической обработке.

Особое значение имеет обработка числовых данных. Значения CSV-файла первоначально считываются как текстовые данные. Для вычисления статистических характеристик значения выбранного пользователем поля необходимо преобразовать в числовой тип. Если отдельное значение не может быть интерпретировано как число, оно не должно участвовать в вычислении числовых характеристик.

Последовательная обработка исходных файлов позволяет сформировать единый набор структурированных данных, используемый для последующей группировки и статистической обработки. Для корректной работы программы исходные CSV-файлы должны иметь одинаковую структуру.

1.3 Алгоритм группировки и статистической обработки данных

Основой разработанного программного средства является алгоритм группировки записей по значению выбранного поля. Задача группировки заключается в разделении исходного набора данных на непересекающиеся группы, каждая из которых объединяет записи с одинаковым значением ключа.

Пусть D – набор исходных записей, а $g(x)$ – функция, возвращающая значение выбранного поля группировки для записи x . Для каждого уникального значения ключа k группа формируется по формуле (2).

$$G_k = \{x \in D \mid g(x) = k\}, \quad (2)$$

где G_k – группа записей с ключом k ;

x – отдельная запись исходного набора;

D – набор исходных записей;

$g(x)$ – функция определения ключа группировки;

k – значение ключа группы.

Таким образом, каждая запись исходного набора относится к группе, соответствующей значению выбранного поля. Например, при выборе столбца «Категория» записи со значениями «Продукты», «Транспорт» и «Развлечения» формируют три отдельные группы.

Для хранения сформированных групп используется структура данных, основанная на сопоставлении ключа группы и списка относящихся к ней записей. При последовательном просмотре исходных данных для каждой записи определяется ключ группировки.

Алгоритм группировки включает следующие действия:

- получение очередной записи исходного набора;
- определение значения выбранного поля группировки;
- поиск группы с соответствующим ключом;
- создание новой группы при отсутствии данного ключа;
- добавление записи в соответствующую группу.

После завершения группировки для каждой группы выполняется обработка значений выбранного числового поля. Пусть множество числовых значений группы k определяется по формуле (3).

$$X_k = \{x_1, x_2, \dots, x_n\}, \quad (3)$$

где X_k – множество числовых значений группы k ;

x_i – i -е числовое значение группы;

n – количество числовых значений группы.

Количество элементов группы вычисляется по формуле (4).

$$N_k = n, \quad (4)$$

где N_k – количество элементов группы k ;

n – количество значений в группе.

Сумма числовых значений группы вычисляется по формуле (5).

$$S_k = \sum_{i=1}^n x_i, \quad (5)$$

где S_k – сумма значений группы k ;

x_i – i -е значение группы;

n – количество значений группы.

Среднее арифметическое значение группы вычисляется по формуле (6).

$$\bar{x}_k = \frac{1}{n} \sum_{i=1}^n x_i, \quad (6)$$

где $\bar{x}_k$ – среднее арифметическое значение группы k ;

x_i – i -е значение группы;

n – количество значений группы.

Для определения медианы значения предварительно располагаются в порядке возрастания. При нечётном количестве элементов медиана вычисляется по формуле (7).

$$Me = x_{\frac{n+1}{2}}, \quad (7)$$

где Me – медиана последовательности;

$x_{\frac{n+1}{2}}$ – центральное значение упорядоченной последовательности;

n – количество значений.

При чётном количестве элементов медиана вычисляется по формуле (8).

$$Me = \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}, \quad (8)$$

где Me – медиана последовательности;

$x_{\frac{n}{2}}$ – первое центральное значение;

$x_{\frac{n}{2}+1}$ – второе центральное значение;

n – количество значений.

Минимальное и максимальное значения группы определяются по формуле (9).

$$x_{\min} = \min(X_k), \quad x_{\max} = \max(X_k), \quad (9)$$

где $x_{\min}$ – минимальное значение группы;

$x_{\max}$ – максимальное значение группы;

X_k – множество числовых значений группы k .

В результате обработки каждой группы формируется сводная запись, содержащая ключ группы, количество элементов, сумму, среднее арифметическое, медиану, минимальное и максимальное значения.

Используемый подход позволяет выполнить группировку за один последовательный просмотр исходных записей. При использовании структуры поиска групп по ключу средняя временная сложность этапа группировки составляет $O(n)$, где n – количество обрабатываемых записей.

Для вычисления медианы требуется сортировка числовых значений внутри каждой группы. В общем случае сложность данной операции составляет $O(n \log n)$. Остальные статистические характеристики могут быть вычислены последовательным просмотром значений группы.

Таким образом, алгоритмическое ядро программы последовательно выполняет объединение исходных записей, формирование групп и вычисление набора статистических характеристик для каждой сформированной группы.

1.4 Структуры данных и сортировка результатов

Выбор структур данных оказывает влияние на организацию алгоритма и удобство обработки информации. Исходная запись CSV-файла представляет собой набор пар «название столбца – значение». Для представления такой записи целесообразно использовать ассоциативную структуру данных, позволяющую получать значение по имени соответствующего поля.

Объединённый набор данных хранится в виде последовательности записей. Такая организация обеспечивает возможность последовательного просмотра данных и добавления записей из нескольких исходных файлов.

Для хранения результатов группировки используется ассоциативная структура, в которой ключом является значение выбранного поля группировки, а значением – коллекция записей или числовых значений, относящихся к данной группе. Пример структуры хранения сгруппированных значений приведён в таблице 1.

Таблица 1 – Структура хранения сгруппированных значений

Ключ группы	Значения
Группа А	12, 18, 25, 30
Группа В	7, 15, 21
Группа С	10, 14, 19, 24

Использование ключа группы обеспечивает получение соответствующей коллекции без последовательного поиска среди ранее сформированных групп. Это упрощает реализацию алгоритма GroupBy и позволяет добавлять новые элементы непосредственно в необходимую группу.

После вычисления статистических характеристик формируется коллекция результатов. Каждая запись содержит название группы и рассчитанные показатели и используется для отображения данных, построения диаграмм и экспорта отчёта.

Для упорядочивания результатов предусмотрена сортировка по названию группы или выбранному статистическому показателю. Пользователь также задаёт направление сортировки. Сортировка выполняется после вычисления статистических характеристик, поскольку рассчитанные показатели могут использоваться в качестве критерия.

2 Обоснование средств разработки и проектирование

2.1 Выбор языка программирования и среды разработки

Для реализации программного средства необходимо выбрать язык программирования, обеспечивающий удобную работу с файлами, поддержку объектно-ориентированного подхода и возможность разработки графического пользовательского интерфейса.

В качестве возможных средств реализации были рассмотрены языки Python, C++ и Java. Сравнение языков программирования по основным критериям приведено в таблице 2.

Таблица 2 – Сравнение языков программирования

Критерий	Python	C++	Java
Работа с CSV-файлами	Модуль csv	fstream, сторонние библиотеки	java.io, сторонние библиотеки
Средства разработки GUI	tkinter, CustomTkinter	Qt, wxWidgets	Swing, JavaFX
Структуры данных	list, dict	vector, map	Collections Framework
Поддержка ООП	Да	Да	Да
Средства тестирования	pytest	Google Test, Catch2	JUnit
Кроссплатформенность	Windows, Linux, macOS	Windows, Linux, macOS	Windows, Linux, macOS
Контейнеризация	Docker	Docker	Docker

Python обладает встроенными средствами работы с CSV-файлами и динамическими структурами данных. Использование списков и словарей позволяет представить строки таблицы, объединённые наборы данных и результаты группировки без большого объёма вспомогательного кода.

C++ и Java также поддерживают объектно-ориентированный подход и разработку графических приложений, однако для рассматриваемой задачи Python обеспечивает более компактную реализацию операций чтения файлов и обработки коллекций. На основании проведённого сравнения Python выбран в качестве языка разработки программного средства.

В качестве среды разработки использован Visual Studio Code. Данная среда обеспечивает редактирование исходного кода, работу со структурой проекта и встроенный терминал для запуска программы, тестов и команд Docker. Поддержка расширений для Python упрощает работу с исходными файлами и поиск ошибок в процессе разработки.

Для управления версиями исходного кода используется система Git. Структура проекта подготовлена для размещения в репозитории, содержащем исходный код, автоматические тесты, тестовые данные, Dockerfile и файл README.md.

2.2 Выбор библиотек и программных средств

При разработке программы использованы стандартные модули Python и внешняя библиотека CustomTkinter. Выбор средств выполнялся с учётом требований индивидуального задания и необходимости сохранения понятной модульной структуры приложения.

Для чтения и записи CSV-файлов используется стандартный модуль csv. Модуль предоставляет средства последовательного чтения табличных записей и формирования словарей, в которых ключами являются названия столбцов. Такой способ представления данных соответствует выбранной структуре обработки записей.

Модуль os применяется при работе с файловыми путями и проверке существования файлов. Для формирования даты и времени создания текстового отчёта используется модуль datetime.

Графический пользовательский интерфейс разработан с использованием библиотеки CustomTkinter. Она предоставляет набор элементов управления, основанных на Tkinter, и позволяет создавать оконные приложения с современным визуальным оформлением. В программе используются окна, панели, кнопки, текстовые подписи, поля выбора и другие элементы управления.

Отдельные стандартные элементы Tkinter применяются через модули tkinter и ttk. Они используются для диалоговых окон выбора файлов, информационных сообщений и табличного представления результатов анализа.

Для автоматического тестирования программного средства используется pytest. Данный инструмент позволяет организовать тесты в отдельных файлах и выполнять проверку отдельных модулей алгоритмического ядра и их совместного взаимодействия.

Контейнеризация приложения выполняется с использованием Docker. Контейнер содержит необходимое программное окружение и зависимости приложения. Для запуска графического интерфейса внутри контейнера используются виртуальный X-сервер и средства удалённого отображения рабочего стола через браузер.

Перечень основных программных средств приведён в таблице 3.

Таблица 3 – Используемые средства разработки

Средство	Назначение
Python	Реализация алгоритмического ядра и логики приложения
CustomTkinter	Разработка графического пользовательского интерфейса
Tkinter и ttk	Диалоговые окна и табличное представление результатов
pytest	Автоматическое тестирование программных модулей
Git	Контроль версий исходного кода
Docker	Контейнеризация и развёртывание приложения
Visual Studio Code	Разработка и редактирование исходного кода

Выбранный набор программных средств соответствует характеру решаемой задачи. Стандартные модули Python используются для файловой обработки. Из дополнительных Python-пакетов применяются CustomTkinter для реализации графического интерфейса и pytest для автоматизированного тестирования.

2.3 Архитектура программного средства

Программное средство разработано с использованием модульной архитектуры. Функции загрузки данных, объединения, группировки, статистической обработки, сортировки, формирования диаграмм и экспорта распределены между отдельными классами и модулями.

Разделение программы на модули уменьшает связанность отдельных частей приложения и позволяет тестировать алгоритмические компоненты независимо от графического интерфейса. Взаимодействие между пользовательским интерфейсом и алгоритмическим ядром координируется главным окном приложения.

Архитектура программного средства включает три основные группы компонентов:

- модули алгоритмического ядра;
- компоненты графического пользовательского интерфейса;
- вспомогательные средства приложения.

К модулям алгоритмического ядра относятся:

- CSVLoader – загрузка и чтение CSV-файлов;
- Merger – объединение данных нескольких файлов;
- GroupByEngine – группировка записей по выбранному полю;
- StatisticsEngine – вычисление статистических характеристик;
- Sorter – сортировка сформированного отчёта;
- ASCIIChartBuilder – формирование текстовых ASCII-диаграмм;
- Exporter – сохранение сформированных результатов в файлы.

Графический интерфейс разделён на самостоятельные панели. К ним относятся панель выбора CSV-файлов, панель настройки параметров анализа, область отображения результатов и строка состояния. Главное окно MainWindow создаёт компоненты интерфейса и связывает действия пользователя с модулями алгоритмического ядра.

Выбранное разделение компонентов позволяет изменять отдельные части программного средства без существенного влияния на остальные модули. Например, изменение способа формирования диаграмм или экспорта результатов не требует внесения изменений в алгоритмы загрузки, группировки и статистической обработки данных. Аналогично, компоненты пользовательского интерфейса используют результаты работы алгоритмического ядра через определённую последовательность вызовов и не содержат непосредственно логики обработки исходных данных.

Последовательность взаимодействия основных модулей представлена в соответствии с рисунком 1.

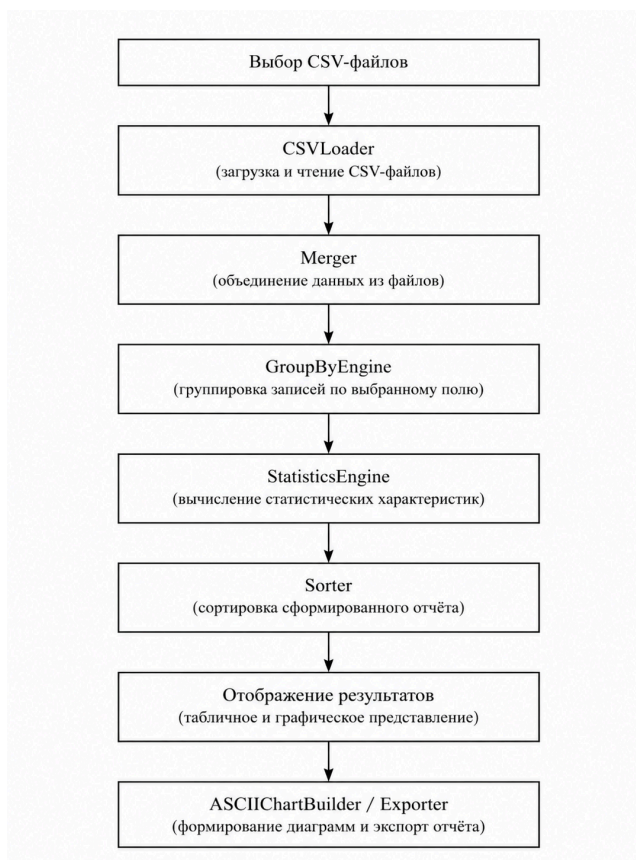


Рисунок 1 – Последовательность взаимодействия основных модулей

После выбора файлов главное окно последовательно передаёт данные модулям загрузки, объединения, группировки и статистической обработки. Полученные результаты сортируются по выбранному критерию и передаются компонентам пользовательского интерфейса для отображения.

При экспорте класс Exporter сохраняет сформированные данные в CSV- или TXT-файл и использует ASCIIChartBuilder для создания текстовых диаграмм. Дополнительный модуль журналирования фиксирует основные операции и возникающие ошибки.

Таким образом, архитектура приложения основана на разделении ответственности между независимыми модулями. Графический интерфейс отвечает за взаимодействие с пользователем, главное окно координирует последовательность обработки, а модули алгоритмического ядра выполняют отдельные операции над данными. Такая организация упрощает тестирование, сопровождение и дальнейшее расширение программного средства.

3 Реализация программного средства

Разработка программного средства выполнена в соответствии с модульной архитектурой, рассмотренной в предыдущем разделе. Реализация включает алгоритмическое ядро, средства формирования и экспорта отчётов, а также графический пользовательский интерфейс.

3.1 Реализация алгоритмического ядра

Алгоритмическое ядро программного средства реализовано набором независимых классов, соответствующих основным этапам обработки данных. Загрузка CSV-файлов выполняется классом CSVLoader, объединение полученных записей – классом Merger, а группировка данных – классом GroupByEngine.

Вычисление статистических характеристик выполняется классом StatisticsEngine. Для каждой сформированной группы определяются количество корректных числовых значений, сумма, среднее арифметическое, медиана, минимальное и максимальное значения. Некорректные или отсутствующие числовые значения исключаются из статистической обработки.

Реализация вычисления статистических характеристик представлена в соответствии с рисунком 2.

```

7  class StatisticsEngine:
10     def calculate(groups, value_field):
28         result = []
29
30         for group_name, rows in groups.items():
31
32             values = []
33
34             for row in rows:
35
36                 try:
37                     values.append(float(row[value_field]))
38                 except (ValueError, KeyError):
39                     continue
40
41             if not values:
42                 continue
43
44             report = {
45
46                 "Группа": group_name,
47
48                 "Количество": len(values),
49
50                 "Сумма": sum(values),
51
52                 "Среднее": sum(values) / len(values),
53
54                 "Медиана": median(values),
55
56                 "Минимум": min(values),
57
58                 "Максимум": max(values)
59
60             }
61
62             result.append(report)

```

Рисунок 2 – Реализация вычисления статистических характеристик

Сортировка сформированного отчёта реализована классом `Sorter`. В качестве критерия могут использоваться наименование группы и рассчитанные статистические показатели. Направление сортировки задаётся пользователем.

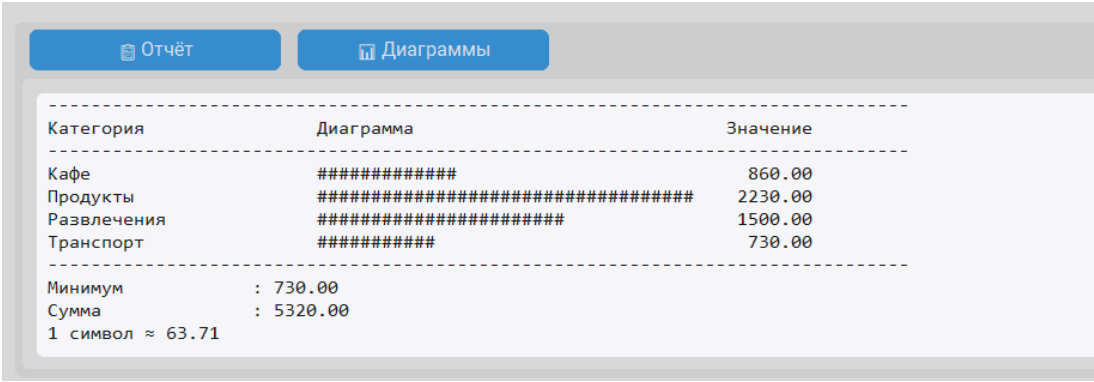
Разделение операций между самостоятельными классами обеспечивает независимое тестирование компонентов алгоритмического ядра и соответствует модульной архитектуре программного средства.

3.2 Реализация формирования диаграмм и экспорта

Для графического представления результатов анализа в программе предусмотрено формирование диаграмм. Пользователь может выбрать горизонтальное или процентное представление результатов.

Текстовые диаграммы формируются классом `ASCIICartBuilder`. В качестве исходных данных используется сформированный сводный отчёт. Для каждой группы определяется величина отображаемого значения и вычисляется длина соответствующей текстовой полосы.

Результат формирования горизонтальной ASCII-диаграммы представлен в соответствии с рисунком 3.



Категория	Диаграмма	Значение
Кафе	#####	860.00
Продукты	#####	2230.00
Развлечения	#####	1500.00
Транспорт	#####	730.00
Минимум	: 730.00	
Сумма	: 5320.00	
1 символ ≈ 63.71		

Рисунок 3 – Результат формирования горизонтальной ASCII-диаграммы

При построении горизонтальной диаграммы длина полосы определяется относительно максимального значения среди всех групп. Это позволяет сохранить пропорции между отображаемыми результатами и ограничить ширину текстовой диаграммы.

Процентная диаграмма отображает долю значения каждой группы в общей сумме. Для каждой группы вычисляется процентное отношение её значения к сумме значений всех групп. Полученное значение используется при формировании текстовой полосы и процентного показателя.

Экспорт сформированного отчёта реализован классом `Exporter`. Результаты могут быть сохранены в форматах CSV и TXT. CSV-файл содержит рассчитанные статистические показатели для каждой группы и формируется средствами стандартного модуля `csv`.

Текстовый отчёт дополнительно содержит дату формирования и ASCII-диаграммы, создаваемые классом `ASCIIChartBuilder`. Перед сохранением выполняется проверка наличия сформированных результатов, а ошибки записи передаются пользовательскому интерфейсу.

3.3 Реализация графического пользовательского интерфейса

Графический пользовательский интерфейс реализован с использованием библиотеки `CustomTkinter`. Главное окно приложения создаётся классом `MainWindow`, который отвечает за размещение компонентов и организацию их взаимодействия с алгоритмическим ядром.

Интерфейс приложения разделён на несколько функциональных областей: панель выбора CSV-файлов, панель параметров анализа, область отображения результатов и строку состояния.

Панель выбора файлов позволяет добавить один или несколько CSV-файлов и очистить список выбранных файлов. Информация о выбранных файлах отображается непосредственно в интерфейсе приложения.

Панель настроек анализа содержит элементы выбора поля группировки, числового поля, критерия сортировки и типа диаграммы. Дополнительно предусмотрен переключатель направления сортировки. Формирование сводного отчёта запускается нажатием соответствующей кнопки.

После запуска обработки главное окно получает список выбранных файлов и последовательно вызывает модули алгоритмического ядра. Загруженные данные объединяются, группируются и передаются модулю вычисления статистических характеристик. Полученный отчёт сортируется в соответствии с выбранными параметрами и сохраняется в текущем состоянии главного окна.

Результаты анализа отображаются в табличной форме. Для каждой группы выводятся наименование группы, количество элементов, сумма, среднее арифметическое, медиана, минимальное и максимальное значения.

Отдельная область интерфейса предназначена для отображения диаграмм. При изменении выбранного типа диаграммы программа обновляет представление результатов на основе уже сформированного отчёта.

Кнопки экспорта позволяют сохранить результаты анализа в CSV- или TXT-файл. Выбор расположения и имени итогового файла выполняется с помощью стандартного диалогового окна файловой системы.

Строка состояния используется для отображения информации о текущем состоянии приложения и результате выполненных операций. Ошибочные ситуации обрабатываются с выводом информационных сообщений пользователю.

Разработанный графический пользовательский интерфейс программного средства представлен в соответствии с рисунком 4.

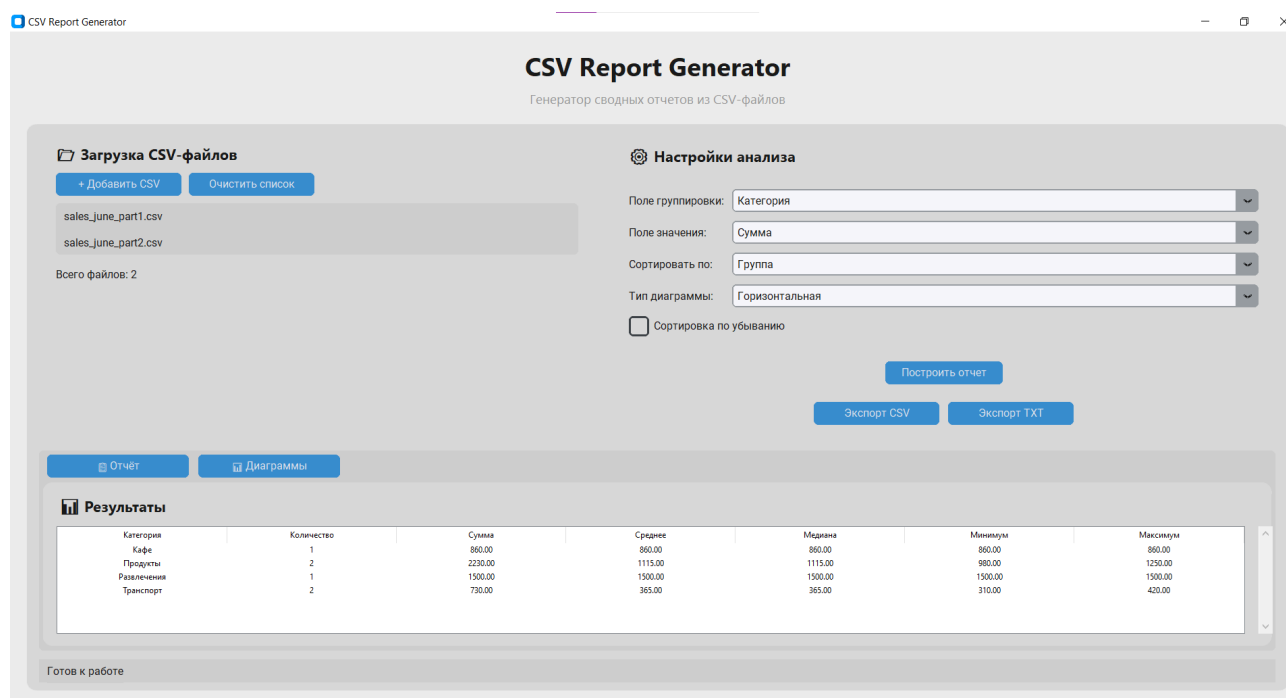


Рисунок 4 – Главное окно программного средства

Таким образом, разработанный пользовательский интерфейс объединяет элементы управления параметрами анализа и средства представления результатов. Главное окно координирует взаимодействие интерфейсных компонентов с модулями алгоритмического ядра, не выполняя непосредственно вычислительные операции.

4 Тестирование и отладка программного средства

4.1 Организация тестирования

Тестирование разработанного программного средства выполнялось с целью проверки корректности алгоритмического ядра, обработки исходных данных и взаимодействия отдельных модулей приложения.

Для автоматизации тестирования использовался фреймворк `pytest`. Тесты размещены отдельно от исходного кода программного средства, что позволяет независимо выполнять проверку реализованных компонентов.

В процессе тестирования проверялись следующие функции программы:

- загрузка и чтение CSV-файлов;
- объединение записей нескольких файлов;
- группировка данных по выбранному полю;
- вычисление статистических характеристик;
- сортировка сформированных результатов;
- формирование ASCII-диаграмм;
- экспорт сформированного отчёта.

Для проверки программного средства использовались тестовые наборы CSV-данных с заранее известными значениями. Это позволяет сравнивать рассчитанные программой статистические характеристики с ожидаемыми результатами.

При тестировании также проверялась обработка отдельных ошибочных ситуаций, включая отсутствие указанного файла.

4.2 Модульное и интеграционное тестирование

Для проверки отдельных компонентов программного средства были разработаны модульные тесты. Тест загрузчика CSV-файлов проверяет корректность чтения исходных данных и обработку ситуации отсутствия указанного файла.

Работа модуля объединения данных проверяется тестами класса `Merger`. В ходе тестирования контролируется последовательное объединение записей нескольких наборов данных в единую коллекцию.

Для модуля группировки проверяется корректность распределения записей по значениям выбранного поля. Тестовые данные содержат несколько записей, относящихся к различным категориям, что позволяет проверить состав сформированных групп.

Отдельные тесты разработаны для модуля статистической обработки. Проверяется правильность вычисления количества элементов, суммы, среднего арифметического, медианы, минимального и максимального значений.

Функционирование модуля экспорта проверяется путём формирования выходного файла и последующего контроля наличия сохранённых результатов.

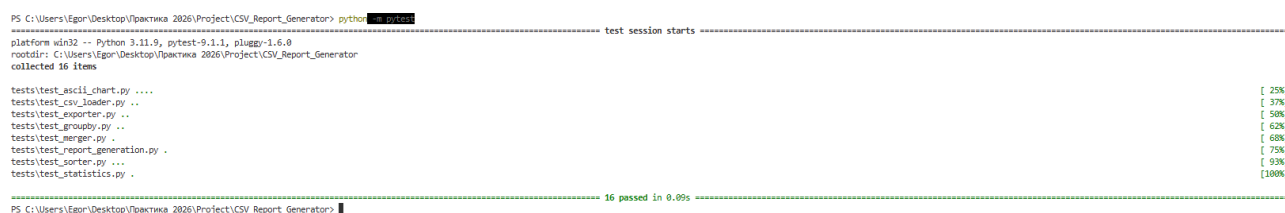
Помимо модульных тестов разработан интеграционный тест формирования сводного отчёта. Он проверяет последовательное взаимодействие основных компонентов алгоритмического ядра при обработке исходных данных. В рамках тестового сценария выполняются загрузка данных, их объединение, группировка и вычисление статистических характеристик.

Таким образом, модульное тестирование позволяет проверить отдельные компоненты программного средства, а интеграционный тест подтверждает корректность их совместного взаимодействия.

4.3 Анализ результатов тестирования

Для запуска автоматизированных тестов использовался фреймворк `pytest`. Тестирование выполнялось из корневого каталога проекта командой `python -m pytest`.

В результате выполнения тестового набора было запущено 16 тестов, охватывающих основные функциональные модули программного средства. Все тесты завершились успешно. Результат выполнения автоматизированного тестирования представлен в соответствии с рисунком 5.



```

PS C:\Users\Egor\Desktop\Практика 2026\Project\CSV_Report_Generator> python -m pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Egor\Desktop\Практика 2026\Project\CSV_Report_Generator
collected 16 items

tests\test_excel_chart.py .... [ 25%]
tests\test_csv_loader.py .. [ 37%]
tests\test_exporter.py .. [ 50%]
tests\test_groupby.py .. [ 62%]
tests\test_merger.py .. [ 68%]
tests\test_report_generation.py . [ 75%]
tests\test_sorter.py ... [ 93%]
tests\test_statistics.py . [100%]

===== 16 passed in 0.09s =====
PS C:\Users\Egor\Desktop\Практика 2026\Project\CSV_Report_Generator>

```

Рисунок 5 – Результат выполнения автоматизированного тестирования

Успешное прохождение всех разработанных тестов подтверждает корректность работы проверяемых компонентов программного средства и их взаимодействия в предусмотренных тестовых сценариях. В ходе тестирования ошибок, препятствующих выполнению основных функций приложения, обнаружено не было.

5 Контейнеризация программного средства

5.1 Подготовка контейнера приложения

Для обеспечения запуска программного средства в изолированном окружении была выполнена контейнеризация приложения с использованием технологии Docker.

Для описания процесса сборки контейнера используется файл Dockerfile, расположенный в корневом каталоге проекта. В качестве базового образа применяется облегчённый образ Python 3.12 slim.

Поскольку разработанное программное средство имеет графический пользовательский интерфейс, в контейнер устанавливаются дополнительные компоненты для организации виртуального графического окружения. Для этого используются Xvfb, x11vnc и noVNC. Компонент Xvfb обеспечивает работу виртуального дисплея, x11vnc предоставляет доступ к графической сессии, а noVNC позволяет взаимодействовать с интерфейсом приложения через веб-браузер.

Зависимости Python устанавливаются из файла requirements.txt. В данном файле указаны библиотека CustomTkinter, используемая для реализации графического интерфейса, и фреймворк pytest, применяемый для автоматизированного тестирования программного средства.

Исходный код и необходимые файлы проекта копируются в рабочий каталог /app. Для запуска контейнера используется сценарий entrypoint.sh. Контейнер предоставляет порт 6080, через который осуществляется доступ к графическому интерфейсу приложения.

5.2 Сборка и запуск программного средства в Docker

Сборка Docker-образа выполняется из корневого каталога проекта. В процессе сборки последовательно создаётся рабочее окружение, устанавливаются системные компоненты и Python-зависимости, после чего исходный код программного средства копируется в контейнер.

Результат успешной сборки Docker-образа представлен в соответствии с рисунком 6.

```
PS C:\Users\Egor\Desktop\Практика 2026\Project\CSV_Report_Generator> docker build -t csv-report-generator .
[+] Building 4.7s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 440B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [internal] load build context
=> => transferring context: 2.27MB
=> [1/8] FROM docker.io/library/python:3.12-slim@sha256:423ed6ab25b1921a477529254bfeeabf5855151dc2c3141699a1bfc8
=> resolve docker.io/library/python:3.12-slim@sha256:423ed6ab25b1921a477529254bfeeabf5855151dc2c3141699a1bfc8
=> CACHED [2/8] WORKDIR /app
=> CACHED [3/8] RUN apt-get update && apt-get install -y python3-tk xvfb x11vnc novnc websoc
=> CACHED [4/8] COPY requirements.txt .
=> CACHED [5/8] RUN pip install --no-cache-dir -r requirements.txt
=> [6/8] COPY . .
=> [7/8] COPY entrypoint.sh .
=> [8/8] RUN chmod +x entrypoint.sh
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:22142209e15e8db6d075bb489cce9d577a4b7b6a8ff762683e382605c541217
=> exporting config sha256:39c441b3c0d6909702aff57e646ab8387cc1e64a8eeb5f14de64d91ad4a8ba1
=> exporting attestation manifest sha256:c11ebd8247505bffc3f35fe4d5e508f213ae4d0e96aa9a1e7f49aa63c52d25ee8
=> exporting manifest list sha256:2ccdd328fab3f1ec89c79ccf1acbf99662bda14d7c9e226b9068180523022b
=> => naming to docker.io/library/csv-report-generator:latest
=> => unpacking to docker.io/library/csv-report-generator:latest
PS C:\Users\Egor\Desktop\Практика 2026\Project\CSV_Report_Generator>
```

Рисунок 6 – Результат сборки Docker-образа

После формирования Docker-образа выполняется запуск контейнера с публикацией порта 6080. Сценарий `entrypoint.sh` осуществляет подготовку виртуального графического окружения и запуск программного средства.

Доступ к пользовательскому интерфейсу выполняется через веб-браузер с использованием `noVNC`. Такой способ запуска позволяет использовать графическое приложение внутри изолированного Docker-контейнера без непосредственного вывода интерфейса средствами операционной системы контейнера.

Результат запуска программного средства в контейнере и доступа к графическому интерфейсу через `noVNC` представлен в соответствии с рисунком 7.

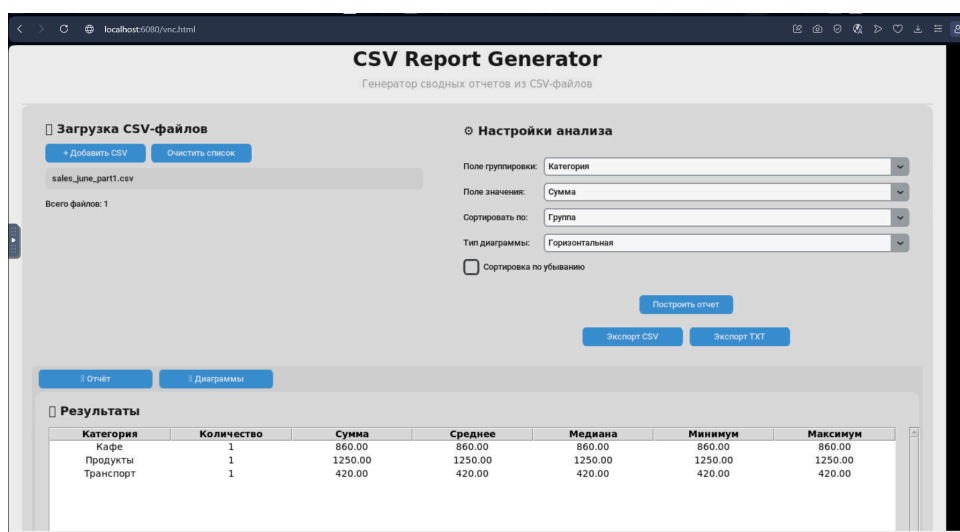


Рисунок 7 – Запуск программного средства в Docker-контейнере через `noVNC`

Использование контейнеризации обеспечивает воспроизводимость программного окружения и автоматическую установку необходимых зависимостей. При наличии установленной среды Docker пользователю не требуется отдельно устанавливать используемые Python-библиотеки и компоненты графического окружения.

6 Руководство пользователя

6.1 Запуск программного средства

Для локального запуска программного средства необходимо наличие установленного интерпретатора Python и требуемых программных зависимостей. Запуск приложения выполняется из корневого каталога проекта. После запуска открывается главное окно программы, содержащее элементы загрузки CSV-файлов, настройки анализа и область отображения результатов.

Программное средство также может быть запущено в Docker-контейнере. В данном случае доступ к графическому интерфейсу осуществляется через веб-браузер с использованием по VNC. Процесс контейнеризации и запуска приложения рассмотрен в разделе 5.

6.2 Работа с программой

Перед формированием отчёта пользователю необходимо загрузить один или несколько CSV-файлов. Добавление файлов выполняется с помощью кнопки «Добавить CSV». Имена выбранных файлов отображаются в области загрузки, а под списком указывается их общее количество.

После загрузки исходных данных пользователь задаёт параметры анализа. В поле группировки выбирается столбец, значения которого используются для формирования отдельных групп записей. В качестве поля значения указывается столбец с числовыми данными, для которого выполняется вычисление статистических характеристик.

Дополнительно пользователь может выбрать критерий сортировки результатов, установить сортировку по убыванию и определить тип текстовой диаграммы.

После настройки параметров необходимо нажать кнопку «Построить отчёт». Программное средство объединяет записи загруженных файлов, выполняет группировку и рассчитывает количество значений, сумму, среднее арифметическое, медиану, минимальное и максимальное значения.

Результаты обработки отображаются в табличном виде в области «Результаты». Для просмотра сформированной текстовой диаграммы используется вкладка «Диаграммы».

6.3 Экспорт результатов

Сформированные результаты могут быть сохранены в форматах CSV и TXT. Для выполнения экспорта используются кнопки «Экспорт CSV» и «Экспорт TXT».

При экспорте в CSV-файл сохраняются рассчитанные статистические характеристики сформированных групп. Полученный файл может использоваться для последующей обработки данных в других программных средствах.

Экспорт в текстовый формат предназначен прежде всего для получения отчёта, пригодного для непосредственного просмотра без использования специализированных средств обработки табличных данных. В отличие от CSV-файла, TXT-отчёт включает не только рассчитанные значения, но и их текстовое и графическое представление в виде ASCII-диаграмм.

TXT-отчёт содержит результаты статистического анализа, а также горизонтальную и процентную ASCII-диаграммы. Результат формирования TXT-отчёта представлен в соответствии с рисунком 8.

```

=====
Дата формирования: 11.07.2026 18:19:17
=====
РЕЗУЛЬТАТЫ АНАЛИЗА
=====

```

Группа	Кол-во	Сумма	Среднее	Медиана	Мин	Макс
Кафе	1	860.00	860.00	860.00	860.00	860.00
Продукты	1	1250.00	1250.00	1250.00	1250.00	1250.00
Транспорт	1	420.00	420.00	420.00	420.00	420.00

```

=====
ГОРИЗОНТАЛЬНАЯ ASCII-ДИАГРАММА
=====

```

```

=====
ASCII REPORT CHART
=====

```

```

Тип диаграммы      : Горизонтальная
Количество групп   : 3
Максимум           : 1250.00

```

Категория	Диаграмма	Значение
Кафе	#####	860.00
Продукты	#####	1250.00
Транспорт	#####	420.00

```

Минимум           : 420.00
Сумма              : 2530.00
1 символ ≈ 35.71
=====

```

```

=====
ПРОЦЕНТНАЯ ASCII-ДИАГРАММА
=====

```

```

=====
ASCII PERCENTAGE CHART
=====

```

```

Тип диаграммы      : Процентная
Количество групп   : 3
Общая сумма        : 2530.00

```

Категория	Диаграмма	Доля
Кафе	#####	33.99%
Продукты	#####	49.41%
Транспорт	#####	16.60%

```

Итого              : 100.00%
1 символ ≈ 2.86%
=====

```

Рисунок 8 – Результат экспорта отчёта в формат TXT

Перед сохранением файла пользователь выбирает каталог и имя выходного файла. После завершения операции сформированный отчёт сохраняется в указанном расположении.

Заключение

В ходе выполнения работы было разработано программное средство для формирования сводных отчётов на основе данных CSV-файлов.

В процессе разработки реализованы загрузка и объединение нескольких CSV-файлов, группировка записей по выбранному полю и вычисление статистических характеристик числовых данных. Для сформированных групп определяются количество значений, сумма, среднее арифметическое, медиана, минимальное и максимальное значения. Реализована сортировка результатов и формирование горизонтальных и процентных ASCII-диаграмм.

Для взаимодействия с программным средством разработан графический пользовательский интерфейс, позволяющий загружать исходные файлы, задавать параметры анализа и просматривать полученные результаты. Предусмотрена возможность экспорта сформированных данных в форматы CSV и TXT.

Модульная структура программного средства обеспечивает разделение операций загрузки, объединения, группировки, статистической обработки, сортировки и экспорта данных. Корректность работы основных программных модулей проверена с использованием автоматизированных тестов на основе фреймворка `pytest`.

Для обеспечения воспроизводимости программного окружения выполнена контейнеризация приложения с использованием `Docker`. Организован запуск графического интерфейса в виртуальном окружении и доступ к нему через веб-браузер с использованием `noVNC`.

Таким образом, поставленная цель разработки программного средства для автоматизации формирования сводных отчётов по данным нескольких CSV-файлов достигнута. Разработанное приложение выполняет предусмотренные требованиями функции и может использоваться для обработки и представления структурированных табличных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Изучаем Python: модуль csv для начинающих с домашним заданием [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/960804/> (дата обращения: 25.06.2026).
- 2 CSV File Reading and Writing [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/csv.html> (дата обращения: 26.06.2026).
- 3 Python Standard Library [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/index.html> (дата обращения: 26.06.2026).
- 4 GUI-приложения с помощью Python-Tkinter [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/otus/articles/903526/> (дата обращения: 28.06.2026).
- 5 CustomTkinter Documentation [Электронный ресурс]. – Режим доступа: <https://customtkinter.tomschimansky.com/documentation/> (дата обращения: 28.06.2026).
- 6 pytest Documentation [Электронный ресурс]. – Режим доступа: <https://docs.pytest.org/en/stable/> (дата обращения: 08.07.2026).
- 7 Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> (дата обращения: 09.07.2026).
- 8 noVNC [Электронный ресурс]. – Режим доступа: <https://novnc.com/info.html> (дата обращения: 09.07.2026).